PRICE PREDICTION AND MARKET ANALYSIS MODULE FOR SUSTAINABLE ANTHURIUM FARMING

Peiris W.V.D

IT22117564

BSc (Hons) in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

PRICE PREDICTION AND MARKET ANALYSIS MODULE FOR SUSTAINABLE ANTHURIUM FARMING

Peiris W.V.D

IT22117564

Dissertation submitted in partial fulfillment of the requirements for the Special Honors Degree of Bachelor of Science in Information Technology Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this report does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or institute of higher learning. To the best of my knowledge and belief, it does not contain any material previously published or written by another person except where acknowledgment is made in the text.

Name	Student ID	Signature
Peiris W.V.D	IT22117564	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the Supervisor: Date:

(Madam Anjalie Gamage)

....

ABSTRACT

The growing of anthuriums (Anthurium spp.), an important part of the ornamental foliage trade, has become an essential livelihood activity for many people in Sri Lanka. However, female and male anthurium growers from all parts of the country experience a lack of adequate income-producing capabilities because there is no reliable market intelligence tool available to them on an ongoing basis. Daily fluctuations in the price of anthuriums have made it difficult for growers to make informed decisions about when and where to sell their products. Additionally, there is no formalized mechanism for forecasting or for evaluating whether current prices provide an advantageous opportunity for sale. As a result, anthurium growers are not taking an informed or systematic approach to marketing their products. This report provides a description of the design, development and evaluation of a module developed to support the development of the price prediction and market analysis capabilities as part of a larger intelligent system designed to enable and promote sustainable practices for the cultivation of anthuriums in Sri Lanka.

A two-level forecast model was developed for this module. In the initial phase, an LSTM model analyses an 8-day sequence of historical prices for a store, variety, and size combination; identifies and stores temporal price patterns corresponding to weekly demand cycles and gradual trend of prices as well as shortest-term price movements; the model then stores a preliminary estimated price as output from the initial level in order to be subsequently entered into phase two of the forecast model which uses an ensemble regression model to combine the temporal representation (temporal price patterns, previously stored), and the contextual present day characteristics of the market in which the store is located (store, variety, units sold, lagged prices, etc) to create a final forecasted price. Prices will be rounded for all products to the nearest half rupee consistent with the customary pricing conventions for Anthuriums sold at retail in Sri Lanka.

This module covers more than just price prediction; it also includes the development of a new three dimension index for this application called the Sustainability, Analysis and Market Intelligence Score (SAMIS). You can think of this as a market scoring index, which calculates the predicted price based on three different statistical perspectives. The Sustainability component determines how reliable and consistent the predicted price will be based on historical statistical data and uses z-score interval logic (based on empirical rule). The Trend component looks at the percentage change between predicted prices and the first price recorded in a 7-day trailing window in order to define the recent price direction (rising/stable/falling).

Lastly, the risk component evaluates the historical volatility associated with the market, by utilizing the Coefficient of Variation (CoV), which is the standard deviation divided by the average price. The final SAMIS score (from 0 - 100) is generated by averaging all three component scores, and is used by the market recommendation engine to categorize the relative market conditions for sale as optimal, favorable, moderate, or unfavorable.

In addition, a module was created that visualizes fluctuations in the market by graphically displaying historical price changes over different time periods or product filters (both can be configured). This visual representation of price changes provides an empirical basis for the assertion that price instability is an inherent and persistent factor in Anthurium Alley market operations and supports the further development of a SAMIS-based assessment methodology. The original REST API was designed to be consumed/deployed via a mobile application developed using React Native; both the data store technology (MongoDB) and deployment management system (Docker) were also utilized. Historical transaction data collected from actual market transactions for Anthuriums in Sri Lanka were used to validate and develop regression-based prediction models upon which the architecture to facilitate the production of stable price forecasts from the same sample market data and use the SAMIS framework for generating comparative market performance analyses based on previously observed market activity can be successfully achieved through the use of hybridized and/or combined predictive models.

Keywords: Anthurium, price prediction, market analysis, SAMIS, market sustainability, time-series forecasting, ensemble regression, deep learning, Sri Lanka, floriculture, decision support

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor and co-supervisor for their continuous guidance, support, and encouragement throughout this research. Their insights and constructive feedback at every stage of the project have been invaluable in shaping this work.

I am also grateful to the Sri Lanka Institute of Information Technology for providing the academic environment and resources that made this research possible. The support of the academic staff in the Department of Information Technology is greatly appreciated.

Special thanks are due to the Anthurium flower market participants and growers in Sri Lanka whose real transaction data formed the foundation of this research. Their cooperation in making market data accessible was essential to the development and validation of the proposed system.

Finally, I wish to extend my heartfelt appreciation to my family and friends for their unwavering support and encouragement throughout the duration of this project.

TABLE OF CONTENTS

DECLARATION OF THE CANDIDATE AND SUPERVISOR ii
ABSTRACT iii
ACKNOWLEDGEMENT iv
TABLE OF CONTENTS 7
LIST OF FIGURES 9
LIST OF TABLES 10
LIST OF ABBREVIATIONS 11
1. INTRODUCTION 12
1.1 Background and Literature Survey 13
1.2 Research Gap 15
1.3 Research Problem 17
1.4 Research Objectives 17
1.4.1 Main Objective 17
1.4.2 Specific Objectives 17
2. METHODOLOGY 19
2.1 Understanding the Key Components of the Research Domain 19
2.1.1 Deep Sequential Learning for Price Forecasting 19
2.1.2 Ensemble Regression Methods 19
2.1.3 Statistical Market Analysis 20
2.1.4 Time-Series Feature Engineering 21
2.2 System Design and Approach 21
2.2.1 Data Collection and Preparation 21
2.2.2 Price Prediction Model Architecture 22
2.2.3 SAMIS Framework Design 24
2.2.4 Market Fluctuation Visualization 29
2.3 Summary of Methodology 29
2.4 High-Level System Architecture 30
2.5 System Requirements 30
2.5.1 Functional Requirements 30
2.5.2 Non-Functional Requirements 31
2.6 Consideration of System Aspects 31
2.6.1 Social Aspects 31
2.6.2 Security Aspects 31
2.6.3 Ethical Aspects 32
2.6.4 Limitations 32
2.7 Commercialization Aspects 32
2.8 Implementation and Testing 33
2.8.1 Code Implementation 33
2.8.2 Testing 39
2.9 Tools and Technologies 47
3. RESULTS AND DISCUSSION 48
3.1 Results 48

3.2 Research Findings and Discussion 49
4. CONCLUSION 51
5. REFERENCES 53

LIST OF FIGURES

Figure 1: data validation
Figure 2: data preprocessing
Figure 3: model initializing and lazy loading
Figure 4: request model
Figure 5: model execution
Figure 6: SAMIS output
Figure 7: examples for fluctuations
Figure 8: examples for fluctuations
Figure 9: examples for fluctuations
Figure 10: examples for fluctuations
Figure 11: model accuracy with fusion
Figure 12: model accuracy without fusion
Figure 13: actual price Vs predicted price graph
Figure 14: Architecture Diagram

LIST OF TABLES

Table 1.2.1: Comparison with existing systems and research gaps identified
Table 2.2.1.1: Dataset fields and descriptions
Table 2.2.2.1: Five-dimensional feature vector constructed at each sequence position
Table 2.2.3.1: Sustainability Component scoring matrix based on z-score intervals
Table 2.2.3.2: Trend Component scoring criteria
Table 2.2.3.3: Risk Component scoring based on Coefficient of Variation
Table 2.2.3.4: SAMIS component score ranges and weights
Table 2.2.3.5: Four-class market recommendation framework
Table 2.3.1: Summary of methodology
Table 2.8.2.1: Dataset split for model training, validation, and testing
Table 3.2.1: Comparison of prediction model configurations and relative performance
Table 3.2.2: SAMIS recommendation accuracy across retrospective test cases

LIST OF ABBREVIATIONS

API	Application Programming Interface
CV	Coefficient of Variation
Docker	Container platform for application deployment
EMA	Exponential Moving Average
JSON	JavaScript Object Notation
LKR	Sri Lankan Rupee
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MongoDB	NoSQL document-oriented database
RF	Random Forest
RMSE	Root Mean Squared Error
REST	Representational State Transfer
SAMIS	Sustainability, Analysis, and Market Intelligence Score
SDK	Software Development Kit
SQL	Structured Query Language

1. INTRODUCTION

Tropical floriculture, which covers the growing of plants in a controlled climate, demands a lot from the ornamental plant Anthurium. The species has a unique shaped flower called a spadix that lasts a long time before wilting, making it very popular in the cut flower market across South and Southeast Asia. Anthuriums are mostly grown in Sri Lanka's Gampaha, Matale, and Kalutara districts where factors such as climate, soil type, and established networks of producers contribute to an industry that is moderately sized and viable economically in rural areas. Long-term profitability for the crop depends on both the agronomic conditions and the day-to-day behavior of the buyers at local Anthurium markets.

The Sri Lankan Anthurium market's structure creates several problems for producers who are trying to make rational sales decisions. Price changes every day because of many factors including consumer demand during festivals, regional oversupply and the presence of buyers at specific collection points and above all buyer influence through informal intermediaries. Monthly flower prices are learned by producers either from their trader contacts over the phone or through direct observation of flower prices in collection centres. By the time producers receive this information, they have made most of their decisions about the timing of harvests and sending their flowers out (dispatch) with only a limited ability to make adjustments thereafter.

Growers currently lack a systematic tool to assess price fluctuations daily. For example, a prediction of a high price is only useful if we know it is due to an increase in demand that will last; in contrast, that same price may not provide a long-term selling opportunity within a consistent pricing environment. Currently, price prediction toolkits in Sri Lanka do not systematically distinguish between prices that are merely temporary and those that represent the expected future market value of a product. For example, determining whether prices are trending up or down each day is an inadequate measure of how the market is performing or what the likely future value of any product will be. This approach does not take into consideration the several components of market risk: historical and statistical price data, momentum directionality, and volatility.

This module provides a solution to these two limitations. It houses a price prediction methodology that utilizes historical transactional data from the Anthurium flower industry to forecast the expected price of a flower by shop, flower type, and flower size. The prediction framework employs a dual layered architecture of deep sequential learning occurrence

extraction (temporal occurrences of prices over an 8-day time frame) integrated with other multidimensional market attributes using ensemble regression to create a price prediction for all flowers sold at any shop.

A structured market sustainability scoring system called SAMIS will provide details of predicted prices in the form of a structured report (i.e., three-component assessment and four-class recommendation) that will help growers determine how sustainable their prices are within a given market. In addition to being able to visualize the market fluctuations, a component (which can be configured to represent the time intervals of price instability) was created to provide graphical representation of price instability, thereby supporting both the motivation of prediction and the functionality of assessing.

The report is organized as follows: (1) A complete background of previous work related to this research is reviewed in Section 1.1; (2) Section 1.2 will define the specific research void being investigated; (3) In Section 1.3, the research problem will be defined; (4) The main and specific objectives will be stated in Section 1.4; (5) All information regarding the methodology will be provided in Section 2 including data preparation, model architecture, SAMIS design, and system implementation; (6) In Section 3, results will be reported and findings will be discussed; and lastly, (7) A summary of the report will be provided in Section 4.

1.1 Background and Literature Survey

For more than thirty years, researchers have attempted to create methods for forecasting agricultural prices. This work can be separated into succession of generations of methods. People primarily relied on classical statistical methods (i.e., Autoregressive Integrated Moving Average [ARIMA] models, plus seasonal versions) from the 1990s through into the 2000s. These methods treat price series as linear combinations of previous values and error terms; they also require linearly transformable price series that are stationary.

However, in the agricultural marketplace, major structural breaks occur frequently due to seasonal behaviours, weather disruption, and demand shocks, resulting in continuing violations of the aforementioned assumptions for the price series as time passes; thus, systematic forecast errors were generated during high economic impact events such as harvest time.

Machine learning techniques yielded a significant enhancement. Support Vector Regression, which finds the best hyperplane that maximizes the margin in a transformed feature space, worked well on small agricultural datasets where linear models did not. Gradient boosting

frameworks like XGBoost and LightGBM became popular in applied agricultural forecasting because they can find complex non-linear relationships and interaction effects without having to explicitly engineer features. Random Forest Regression builds a group of decision trees that grow on their own and combines their predictions. It has similar non-linear modeling abilities and is naturally resistant to overfitting thanks to bootstrap sampling and random feature selection. Several studies on predicting vegetable and grain prices found that Random Forest and gradient boosting methods always did better than ARIMA and linear regression baselines, especially on datasets with seasonal patterns that weren't regular.

However, one basic drawback associated with this kind of approach is the assumption that price records are treated as independent cross-sectional data sets rather than time series data. One can manually create lag variables and include them as part of the input data set; however, the algorithm does not provide any facility to capture the sequence of the values in any way. This is particularly important in the case of markets wherein the forecasting capacity of historical price records not only depends on their values but also on their evolution.

Long Short-Term Memory (LSTM) networks were created to handle the issue of sequential dependence. In an LSTM network, time series inputs are processed, one step at a time, and a cell state is kept to propagate the learned information across the entire time-series input sequence using learned gating functions. Each time step has three gates (the forgetting gate, the input gate and the output gate) to figure out what past signals can be kept in memory and which ones are just noise, thus allowing an LSTM to be able to detect complicated temporal patterns that cannot be detected by a traditional stationary (non-time-series) model. The analyses of LSTM's performance with agricultural price series illustrate the LSTM has performed much better than the statistical and non-time-series approaches used to date to analyze commodities with distinct weekly and seasonal patterns.

The realization that sequential and ensemble models tackle distinct facets of the forecasting challenge resulted in the creation of hybrid architectures. In a standard sequential fusion design, a recurrent model like an LSTM is used to create a price representation that takes time into account. This representation is then added as a feature to an ensemble or regression model that makes the final forecast. This method takes advantage of the strength of deep sequence models for finding temporal patterns and the strength and flexibility of ensemble methods for finding patterns in different contexts. Research utilizing LSTM-ensemble fusion for commodity price forecasting has indicated cumulative enhancements beyond the performance of individual components, particularly during phases of significant market volatility.

Price forecasting specific to the floriculture domain remains relatively underdeveloped compared to the more general agricultural forecast literature. Research into the markets of cut flowers is done mostly in relation to export auctions based in the Netherlands and Kenya, places where significant amounts of transaction data exist. Most such research indicates the usefulness of machine learning for predicting flower prices, though such research is performed on a wholesale and export basis, which is quite different from the retail/semi-wholesale nature of the Anthurium market in Sri Lanka. To the best of the author's knowledge, there exists no price forecasting system designed specifically for the Anthurium market of Sri Lanka, nor for the floriculture industry in general paired with sustainability scoring.

The topic of sustainability evaluation in relation to agriculture markets has hardly been addressed by any academic literature. The existing techniques for the evaluation of market risk in agriculture markets use simple measures of volatility, such as standard deviations or historical price range ratios. These metrics were used merely as descriptive statistics instead of being included in a framework that can guide decision-making. The Coefficient of Variation metric, that is calculated by dividing standard deviation by average price and provides a scale-free measure of price variability, has been utilized in commodity markets; however, no systematic inclusion of the CV as a scoring technique for small farmers has been proposed yet. Z-score based techniques for statistical analysis that rely on interval logic and the Empirical Rule are well-established methods for financial and quality assurance analysis; however, they were never used as part of an agricultural market sustainability index construction. SAMIS developed in this research is the first formally defined market sustainability index for the ornamental floriculture sector known to the author.

1.2 Research Gap

Several particular gaps are addressed by this research, on review of literature and available tools. These fall into both the technical aspect of price prediction and the market decision support to farmers domain.

The first and most pertinent gap is the almost non-existence of quantitative price prediction tools for the floriculture market of Sri Lanka. Though there have been many applications of machine learning techniques in the sphere of agricultural price forecasting of staple crops, none has been designed for, or used, in the domain specific context of the Anthurium market. There have been few relevant studies in existing literature, and as the price of Anthurium is influenced not only by demand and supply, but also by the variety and size grading of the product, as well

as local specific characteristics, and prices are traditionally set at intervals of 0.5 rupees, existing generic models for the forecasting of agricultural prices would require significant domain specific modification.

Secondly, existing price forecasting tools often deal with a one-dimensional prediction, in the form of predicted prices. However, agricultural price forecasting research generally ends at generating predicted prices, and gives no consideration to what those predicted prices mean in the historical context of the market. For example, a predicted price which is an anomaly relative to historically predicted values carries vastly different implications for the selling decision of a farmer, than does an otherwise equivalent predicted price in a typically trending and volatile market. The problem of assessing agricultural price predictions along various dimensions of "market quality" has never been formally investigated.

Thirdly, no market fluctuation visualization tools currently exist for Anthurium price data. It has not yet been empirically demonstrated that market fluctuation is a feature of Anthurium prices in Sri Lanka, rather than an anomaly. This should be displayed with a graphical characterization of price fluctuation patterns in Sri Lanka.

Study / System	Domain	Hybrid Model	Sequential Modeling	Floriculture	Sustainability Score	Sell Recommendation
ARIMA methods	Staple crops	No	Partial	No	No	No
SVR / RF methods	Vegetables, grains	Partial	No	No	No	No
LSTM standalone	Commodity	No	Yes	No	No	No
LSTM-XGBoost fusion	Commodity	Yes	Yes	No	No	No
Cut flower studies	Export floriculture	No	Partial	Yes	No	No
CV-based risk tools	Commodity	No	No	No	Partial	No
This Research (Proposed)	**Anthurium, Sri Lanka**	**Yes**	**Yes**	**Yes**	**Yes (SAMIS)**	**Yes (4-class)**

Table 1.2.1: Comparison with existing systems and research gaps identified

1.3 Research Problem

The central research problem being solved by this module is: Sri Lankan Anthurium growers make daily selling decisions without any prediction of market price, without a structured method for deciding if an actual or expected price presents an economically beneficial market opportunity with statistical stability, and without a representation of actual historical market volatility which is the source of such decision risk.

This research problem has three specific aspects. One is informational; it regards the absence of predictions available to growers at the time that decisions on harvesting and shipping must be made. A second is interpretive, it relates to the grower's ability, given an actual or forecasted price, to know if the market is at a historically high or low price level, whether the market will sustain the indicated price, whether it's a normal spike or the beginnings of a market trend. The third is evidential, regarding the lack of a readily interpretable demonstration that Anthurium prices are structurally volatile – a statement which motivates this entire solution – in that there is no Anthurium-market specific tool to help present such evidence to the growers.

A solution addressing these three aspects has to be a complete, connected system moving from raw data, to prediction, to an informed decision, to a recommendation, each step tailored to the problem domain.

1.4 Research Objectives

1.4.1 Main Objective

The primary objective of this research is to develop, implement, and evaluate an integrated price prediction and market analysis module for the Anthurium floriculture sector in Sri Lanka, capable of generating daily price forecasts and producing structured market sustainability assessments and selling recommendations accessible through a mobile application.

1.4.2 Specific Objectives

1. To collect and prepare a real transaction dataset from Sri Lankan Anthurium markets capturing daily price, variety, size, shop, and demand information for use in model training and evaluation.
2. To design and implement a hybrid sequential ensemble price prediction architecture that combines temporal price pattern extraction with contextual market feature integration for superior prediction stability.

3. To develop the SAMIS framework : a three component market sustainability scoring index using z score interval logic, percentage change momentum analysis, and Coefficient of Variation risk assessment and implement a four class selling recommendation engine driven by SAMIS component values.
4. To implement a market fluctuation visualization module that generates graphical representations of historical Anthurium price movement to provide empirical evidence of market price instability.
5. To deploy all components as a REST API integrated with a React Native mobile application, enabling real time price prediction and SAMIS assessment for growers in the field.
6. To evaluate the prediction architecture against single model baselines and validate the SAMIS framework against retrospective market outcomes.

2. METHODOLOGY

2.1 Understanding the Key Components of the Research Domain

The price prediction and market analysis module rests on three technical foundations: deep sequential learning for temporal price pattern extraction, ensemble regression for robust final prediction, and statistical market analysis for the SAMIS scoring framework. Each is described below.

2.1.1 Deep Sequential Learning for Price Forecasting

Anthurium price patterns can be described as having characteristics that are ideal for modeling sequences. Patterns such as weekly cycles that vary depending on which day of the local market week it is; smooth trends that change over the seasons as determined by cultivation cycle and event calender; as well as intermittent jumps up or down due to supply-side interruptions and festival induced jumps up in demand, can all be characterized as temporal dependencies; where there is a relationship between tomorrow’s price and that of recently observed history which helps explain tomorrow's price.

The network architecture that was chosen, Long Short-Term Memory was designed to be able to learn these dependencies from sequential data. It differs from simple recurrent networks, by using three gating mechanisms; a forget gate which allows it to determine how much to retain from the previous cell state; an input gate that decides what to take out of the new input, and an output gate to determine what to output to the network; to selectively process its history. This means that it is able to hold onto important long-term dependencies, while filtering out less important short-term variations; a trait which is particularly valuable for markets such as that of Anthuriums, where the most useful information may only become apparent over a number of days.

The LSTM uses a sequence length of eight days to produce its forecasts, where each day is a 5-D input of the day’s price, three lag variables, and an exponentially weighted moving average. The decision of eight days was made because typically weekly cycles are observable within a week for Anthurium market data, and also because certain combination types do not trade at every opportunity, so a slightly longer time sequence is needed.

2.1.2 Ensemble Regression Methods

The multiple independent model predictions are averaged together in ensemble regression methods, which will create a lower variance for the target variable and therefore produce better

generalization. Since Random Forest Regression is an ensemble of multiple decision trees created by using bootstrap sampling of the training data and using a random sample of the features to create each decision tree node, the Random Forest technique is a good choice for modelling the context-specific features related to Anthurium market prediction. Many of the contextual features related to Anthurium market prediction are categorical (or very close to being categorical) and represent meaningful differences in the market — for example, shops are selling to different buyer types with unique quantity demands, flower varieties can have structurally different price levels relative to each other, and supply and demand combinations affect pricing.

These different features all interact with one another and the sequential price signal in non-linear ways that cannot be identified with linear regression models. The decision tree structure of the Random Forest can naturally accommodate these interactions, without requiring the use of explicit polynomial transformations or explicit feature combinations. Ensemble averaging from each of the constituent decision trees will suppress the degree to which each individual transaction from the market is considered an outlier; therefore Random Forest will produce a much more stable prediction for Anthurium market prices in a market that occasionally has extreme price transactions.

2.1.3 Statistical Market Analysis

SAMIS' statistics comprise mean price and standard deviation from the historical price distribution, percentage change from predicted price to most recent reference price, and the Coefficient of Variation of historical prices. The three measures represent three independent facets of market quality.

The mean and standard deviation contribute to the Sustainability Component, which tests predicted price using z-score intervals based on the Empirical Rule, stating that for a normal distribution, approx. 68% of observations will fall within 1 standard deviation from the mean and approx. 95% will fall within 2 standard deviations. Applying this rule to the historical distribution, it becomes possible for the system to declare a predicted price statistically ordinary or peculiar in the context of the historical market and thus a genuine or apparent price opportunity.

The Coefficient of Variation (standard deviation/mean) contributes to the Risk Component, since it accounts for the price level in defining price volatility and thus makes comparisons between varieties and size grades of differing mean price meaningful. A 200 LKR market with standard deviation 16 LKR is no more volatile than a 100 LKR market with standard deviation 8 LKR, and should receive identical risk classification. A raw standard deviation would declare the 200 LKR market more volatile.

2.1.4 Time-Series Feature Engineering

The sequential feature construction for the LSTM input draws on two core techniques from time-series analysis. Lag features the price recorded at a specific number of days prior to the current position capture both short-term momentum (lag-1, lag-2) and medium term cyclical patterns (lag-4). The inclusion of lag-4 is motivated by the typical four to five day trading frequency observable in local Anthurium markets, where prices often exhibit mid week cycle effects.

The Exponential Moving Average with span four is included as a noise-reduced representation of recent price trend direction. Unlike a simple moving average, the EMA applies exponentially decaying weights to past observations, giving higher importance to more recent prices while retaining information from earlier in the window. With span four, the smoothing factor alpha = 0.4 provides a balance between responsiveness to genuine trend changes and resistance to single-day price spikes.

2.2 System Design and Approach

The module was designed as a sequence of functional stages. Data preparation feeds into price prediction, which in turn feeds into SAMIS scoring, which drives the recommendation output. The fluctuation visualization component operates as a parallel analytical module drawing on the same underlying dataset.

2.2.1 Data Collection and Preparation

Transaction data was collected from multiple Anthurium flower retail shops across Sri Lanka. Each record captured the following fields: transaction date, shop identifier, flower variety, size grade, seller price per stem in LKR, and units sold on that day. The data collection process involved direct engagement with market participants and systematic recording of daily transaction logs over an extended period.

Prior to use in model training and SAMIS computation, the data underwent a structured preparation process. Date fields were parsed using day-first formatting to ensure consistency across sources with varying date notation conventions. Categorical fields including shop, variety, and size were normalized to lowercase with leading and trailing whitespace removed, ensuring that variations in capitalization or spacing across data sources did not create spurious product combination splits. Records with missing or unparseable date values were excluded. Records with missing price values were similarly excluded, as price imputation would introduce artificial patterns in a dataset where the price field is both the target variable and a feature.

Field	Data Type	Description
date	Date (DD-MM-YYYY)	Transaction date, parsed day-first
shop	String (normalized)	Shop identifier - encodes location and buyer demographics
variety	String (normalized)	Anthurium variety - encodes demand and pricing tier
size	String (normalized)	Flower size grade - small, medium, large
seller_price_per_stem	Float (LKR)	Daily seller price per stem - target variable and key feature
units_sold	Float	Daily demand volume - proxy for market demand strength

Table 2.2.1.1: Dataset fields and descriptions

For product combinations with fewer than eight trading days in the historical record prior to a target date, a forward-fill padding strategy was applied: the earliest available price for that combination was repeated to fill the remaining positions in the sequence. This preserves the uniform (8, 5) input tensor shape required by the sequential model without introducing artificial directional signals, since the padded values are all identical.

2.2.2 Price Prediction Model Architecture

The price prediction architecture operates in two sequential stages. The first stage extracts a temporally informed price estimate from the eight-day price sequence using a deep sequential learning model. The second stage combines that estimate with contextual market features and refines it using an ensemble regression model to produce the final price forecast.

For each position i in the eight day window, with the price sequence ordered chronologically as {p_0, p_1, ..., p_7}, a five dimensional feature vector is constructed as follows:

Feature	Notation	Description
Current price	*p*	Recorded seller price per stem at position i
Lag-1	*p(i-1)*	Price one day prior : captures immediate price momentum
Lag-2	*p(i-2)*	Price two days prior : captures short-term trend direction
Lag-4	*p(i-4)*	Price four days prior : captures mid-week market cycle effects
EMA-4	*EMA4(p[0:i])*	Exponential moving average with span 4 : smoothed trend representation

Table 2.2.2.1: Five dimensional feature vector constructed at each position of the eight day price sequence

Exponential Moving Average

The Exponential Moving Average is computed recursively with smoothing factor $\alpha = 2 / (\text{span} + 1)$. For span 4, $\alpha = 0.4$:

$$EMA_t = 0.4 \cdot p_t + 0.6 \cdot EMA_{t-1}$$

This gives higher weight to more recent observations while retaining the influence of earlier prices in the window. The full eight-step sequence tensor of shape (8, 5) is passed as input to the sequential learning model.

Sequential Model (LSTM)

The sequential model processes this tensor step by step. At each time step *t*, the LSTM cell updates its internal state through three gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(input gate)}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(output gate)}$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad \text{(candidate cell state)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(cell state update)}$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{(hidden state output)}$$

where σ denotes the sigmoid activation function, ⊙ is element-wise multiplication, and W_*, b_* are learned weight matrices and bias vectors. The final hidden state h_8, produced after

processing all eight time steps, is projected through a dense output layer to produce the preliminary price estimate lstm_pred.

Feature Vector & Ensemble Model

This lstm_pred value is then assembled into a structured feature vector alongside contextual market attributes and statistical features:

$$\mathbf{x}_{RF} = [\text{shop, variety, size, lstm_pred, units_sold, } \text{lag}_1, \text{lag}_2, \text{lag}_4, \text{EMA}_4]$$

Categorical features are encoded through a pre trained preprocessing pipeline. This vector is passed to the ensemble regression model, which generates the final price prediction by averaging across B independently trained decision trees:

$$p_{\text{final}} = (1/B) \cdot \sum^{B}_{b=1} T_b(\mathbf{x}_{RF})$$

The predicted price is finally rounded to the nearest half rupee increment, consistent with the discrete pricing conventions observed in Sri Lankan Anthurium markets:

$$p_{\text{rounded}} = \text{round}(p_{\text{final}} \cdot 2) / 2$$

This two-stage design is deliberate. The sequential model cannot incorporate categorical market attributes efficiently , it is architecturally optimized for ordered sequences, not feature tables. The ensemble model, by contrast, cannot learn temporal dependencies from raw lag features as effectively as a model designed for sequence processing. By using each component for what it does best, and combining their contributions through the feature fusion, the architecture achieves greater predictive stability than either component produces independently.

2.2.3 SAMIS Framework Design

SAMIS : Sustainability, Analysis, and Market Intelligence Score is a composite market scoring index developed specifically for this research. It converts the predicted price into a structured three component assessment of market quality and drives a four-class selling recommendation. The composite score is bounded in [0, 100] and computed as:

$$\text{SAMIS} = S(p, \mu, \sigma) + T(p, p_{\text{recent}}) + R(\sigma, \mu)$$

The three components are computed independently. A weakness on one dimension does not affect the scores of the other two, and the recommendation engine responds to the specific pattern of component values rather than treating the composite total as a sufficient statistic. The

statistical foundations for all three components are the mean price μ and standard deviation σ of the historical price series for the same shop/variety/size combination:

$$\mu = (1/n) \cdot \sum_{i=1}^{n} p_i$$

$$\sigma = \sqrt{[(1/n) \cdot \sum_{i=1}^{n} (p_i - \mu)^2]}$$

Sustainability Component (S)

The Sustainability Component evaluates the statistical plausibility of the predicted price relative to the historical price distribution. The underlying question is whether the predicted price represents a continuation of normal market behavior or a statistically unusual event. This matters practically because a high predicted price may reflect genuine demand growth or may simply be a transient anomaly. A grower who cannot distinguish between these cases is likely to overreact to the former and underreact to the latter.

The assessment uses z-score interval logic grounded in the Empirical Rule. Under an approximately normal distribution, roughly 68% of historical prices fall within one standard deviation of the mean, and approximately 95% fall within two standard deviations. Predicted prices beyond the two-standard-deviation boundary are classified as statistical outliers in either direction an unusually high predicted price is treated with the same caution as an unusually low one, since both indicate atypical market conditions that may not persist.

Condition	Score	Classification	Interpretation
$\mu - \sigma \leq p \leq \mu + \sigma$	40	Highly Sustainable	Price within the central 68% of historical distribution normal market behavior
$\mu - 1.5\sigma \leq p \leq \mu + 1.5\sigma$	30	Sustainable	Moderate deviation from mean; within expected range
$\mu - 2\sigma \leq p \leq \mu + 2\sigma$	20	Moderately Sustainable	Approaching statistical boundary; unusual but not extreme
$p \notin [\mu - 2\sigma, \mu + 2\sigma]$	10	Not Sustainable	Statistical outlier; predicted price reflects transient or anomalous market conditions

Table 2.2.3.1: Sustainability Component scoring matrix based on z-score interval analysis

The scoring ceiling of 40 for this component : higher than the 30 ceiling assigned to each of the other two components reflects the interpretive priority placed on statistical price normality. A market whose predicted price is statistically representative of historical behavior is

fundamentally more reliable as a selling environment than one where the price is rising or the market is stable but the predicted price is a statistical outlier.

Trend Component (T)

The Trend Component measures the directional momentum of the predicted price relative to recent market activity. Where the Sustainability Component evaluates the predicted price against the full historical distribution, the Trend Component focuses specifically on the short-term price trajectory whether the market is moving in a direction that favors selling.

Let p_{recent} denote the first price of the trailing seven-day window. The percentage change between this reference price and the predicted price is:

$$\Delta\% = (p - p_{recent}) / p_{recent} \times 100$$

Condition	Trend Label	Score	Practical Meaning
$\Delta\% > 3\%$	Rising	30	Price appreciation above noise threshold favorable momentum for selling
$-3\% \leq \Delta\% \leq 3\%$	Stable	15	Price in equilibrium no strong directional signal
$\Delta\% < -3\%$	Falling	5	Market contracting deferring where possible may recover value

Table 2.2.3.2: Trend Component scoring criteria and practical interpretation

The +/-3% tolerance band around the stable zone is a deliberate design choice. Day to day price variation of up to 3% is common in Anthurium retail markets and does not reliably signal a directional trend it is more accurately characterized as market noise arising from natural demand variation. Classifying every sub-3% movement as a meaningful trend would generate unstable, frequently reversing recommendations that growers could not act on reliably. The band ensures that only sustained, economically meaningful price movements trigger Rising or Falling classifications.

Risk Component (R)

The Risk Component evaluates the reliability of the market as a whole, independently of where the current predicted price falls or which direction it is moving. It asks: how predictable and stable has this market historically been? A market with tightly clustered price history is one where predictions carry relatively high confidence and selling decisions can be made with less hesitation. A market with wide historical price swings is one where even an accurate predicted

price may be followed by a sharp reversal, effectively increasing the grower's exposure regardless of the prediction's technical accuracy.

The Coefficient of Variation captures this distinction in a scale-independent way:

$$CV = \sigma / \mu$$

CV Range	Risk Level	Score	Market Interpretation
CV < 0.05	Low	30	Highly stable market : price fluctuations are minor relative to the mean; forecast confidence is high
0.05 <= CV <= 0.10	Medium	20	Moderate volatility : reasonable confidence in forecast with some caution warranted
CV > 0.10	High	10	Volatile market history : predicted price should be treated with reduced confidence

Table 2.2.3.3: Risk Component scoring based on the Coefficient of Variation

The CV thresholds of 0.05 and 0.10 represent meaningful boundaries in the floriculture retail context. A CV below 0.05 implies that prices rarely deviate by more than 5% of the mean , a condition consistent with stable consumer demand and predictable supply. A CV above 0.10 implies that price swings of more than 10% of the mean are common, indicating structural market instability where external shocks occur frequently enough to materially reduce forecast reliability.

Composite SAMIS Score

The three components are summed to produce the composite SAMIS score:

$$SAMIS = S + T + R, \text{ with } 0 <= SAMIS <= 100$$

Component	Min Score	Max Score	Weight	Statistical Basis
S : Sustainability	10	40	40%	z-score interval / Empirical Rule
T : Trend	5	30	30%	7-day percentage price change
R : Risk	10	30	30%	Coefficient of Variation
SAMIS Total	**25**	**100**	**100%**	**Composite index**

Table 2.2.3.4: SAMIS component score ranges, relative weights, and statistical foundations

The minimum achievable score of 25 represents a market where the predicted price is a statistical outlier, the recent price trend is falling, and historical volatility is high simultaneously. This floor is intentionally non zero : even in severely unfavorable conditions, some market information remains. The maximum of 100 represents a market where the predicted price is fully within the one-standard-deviation historical range, a rising trend is confirmed above the 3% threshold, and historical volatility is below the 0.05 CV threshold.

Four-Class Market Recommendation

The SAMIS score is evaluated through a four-class recommendation framework that uses both the composite score and individual component classifications. This prevents situations where a single strong component masks serious weaknesses in the others.

Class	SAMIS Range	S Score	T Score	R Score	Recommendation
1 Optimal	85 - 100	40	30	30	Sell immediately - all conditions favorable
2 Favorable	65 - 84	>= 30	>= 15	>= 20	Recommended to sell - positive overall market outlook
3 Moderate	40 - 64	20	Any	Any	Monitor and consider partial sale - mixed signals present
4 Unfavorable	< 40	10	5	10	Defer selling if post-harvest storage permits

Table 2.2.3.5: Four-class market recommendation framework derived from SAMIS component scores

Class 1 (Optimal) requires all three components at their maximum values simultaneously : predicted price within one standard deviation, rising trend above 3%, and CV below 0.05. This triple conjunction ensures the highest recommendation tier is reserved for genuinely favorable markets, not those that are strong on one dimension and weak on others. Class 4 (Unfavorable) can be triggered by the composite score falling below 40, by the Sustainability Component reaching its minimum of 10, or by both the Trend and Risk Components simultaneously registering their minimum scores meaning any one of these conditions is sufficient to generate a protective recommendation.

2.2.4 Market Fluctuation Visualization

The fluctuation visualization module was developed to provide empirical evidence that Anthurium price instability is a structural feature of the market, not an occasional exception. Without this evidence, the motivation for both the prediction system and the SAMIS framework rests on assertion rather than demonstration. The module accepts a date range, flower variety, and optional filters for shop and size, then retrieves all matching transaction records within that window, computes daily average prices across the filtered records, and generates a time-series line graph of the resulting price trajectory.

Graphs are generated server-side using the Matplotlib library configured in non-interactive mode, rendered to PNG format, and returned to the mobile client as base64-encoded strings embedded in the API response. This approach avoids file system dependencies and is compatible with containerized deployment. The resulting graphs allow both growers and analysts to observe the historical pattern of price movements whether prices exhibit regular weekly cycles, sustained trends, or irregular spikes providing a visual foundation for understanding why a structured prediction and assessment system is needed.

2.3 Summary of Methodology

Research Component	Methods and Algorithms Used	Expected Output	Novelty Contribution
Price Prediction	LSTM + Ensemble Regression sequential fusion; EMA; lag feature engineering	Daily price per stem in LKR (rounded to 0.5)	Hybrid fusion for floriculture market : first for Sri Lankan Anthurium
SAMIS Scoring	z-score interval logic; percentage change; Coefficient of Variation	SAMIS score 0-100 with three component labels	Novel market sustainability index : first of its kind for floriculture
Recommendation Engine	Rule-based four-class multi-condition classification from SAMIS components	Optimal / Favorable / Moderate / Unfavorable class with rationale	Actionable selling guidance from statistical market assessment
Fluctuation Visualization	Time-series daily average aggregation; Matplotlib graph generation; base64 encoding	PNG graph embedded in API response	First graphical price instability tool for Sri Lankan Anthurium markets

Table 2.3.1: Summary of methodology : components, methods, outputs, and novelty contributions

2.4 High Level System Architecture

The system is organized as a layered architecture. The data layer consists of the MongoDB database storing all transaction records accessed and updated through the mobile application. The processing layer consists of the FastAPI backend service, which loads the pre trained prediction model artifacts at startup, exposes REST API endpoints for price prediction and fluctuation analysis, and performs all SAMIS computations on each prediction request. The presentation layer is the React Native mobile application, which sends prediction requests with date, shop, variety, and size parameters and displays the returned price, SAMIS score with component breakdown, recommendation class, and fluctuation graphs to the grower.

Docker containers manage the deployment of the backend service, ensuring consistent execution environments across development, testing, and production. Model artifacts : the trained sequential model in Keras format, the trained ensemble model as a serialized pickle file, and the preprocessing pipeline are loaded from persistent storage at container startup and held in memory throughout the service lifecycle, enabling sub-second response times for price prediction requests.

2.5 System Requirements

2.5.1 Functional Requirements

7. The system predicts the daily seller price per stem in LKR for a specific shop, flower variety, and size grade on a requested date.
8. The system computes a SAMIS score for each price prediction, providing individual Sustainability, Trend, and Risk component scores alongside their labels.
9. The system classifies each prediction into one of four market recommendation classes and returns the recommendation to the mobile application.
10. The system generates historical price fluctuation graphs for any combination of date range, variety, and optional shop and size filters on request.
11. The system handles product combinations with limited historical data using a forward fill padding strategy that avoids introducing artificial trends.
12. The system exposes all functionality through a REST API accessible from the React Native mobile application.
13. The system provides a health check endpoint reporting model load status, data date range, and record count.

2.5.2 Non Functional Requirements

14. Price prediction response time should be under 500 milliseconds on standard server hardware, supporting real time use in the field.
15. The prediction pipeline must be deterministic for a given input the same shop, variety, size, and date must always return the same price and SAMIS score.
16. The system must handle requests for product combinations with zero historical data gracefully, falling back to dataset wide mean price with appropriate SAMIS scoring.
17. All categorical input fields must be case and whitespace-insensitive to accommodate variation in user input.
18. The API must return structured JSON responses that the mobile application can parse and display without additional processing.
19. The Docker deployment must be reproducible model artifacts must be version controlled alongside deployment configuration.

2.6 Consideration of System Aspects

2.6.1 Social Aspects

The primary beneficiaries of this system are small and medium-scale Anthurium growers in Sri Lanka who currently make selling decisions without any quantitative market information. By providing daily price predictions and structured market assessments through a mobile application, the system reduces the information asymmetry that currently places producers at a disadvantage relative to buyers and intermediaries. The SAMIS recommendation framework is designed to be interpretable without requiring statistical knowledge a grower receiving a Class 1 (Optimal) recommendation knows to sell, while a Class 4 (Unfavorable) recommendation signals caution, without needing to understand the underlying z-score or CV calculations. The system is accessible to anyone with a smartphone, regardless of prior technical knowledge.

2.6.2 Security Aspects

All API communications between the mobile application and the backend service occur over HTTPS. The MongoDB database enforces role based access control, with the backend service using credentials that grant read and write access only to the transaction data collection. The containerized deployment isolates the service from the host system and prevents unauthorized access to model artifacts or raw data. User authentication in the mobile application ensures that

price queries are associated with authenticated accounts, preventing anonymous abuse of prediction requests.

2.6.3 Ethical Aspects

The market data used to train and evaluate the prediction models was collected from real market participants with appropriate consent. The system does not store or expose individual grower transaction data to other users. The SAMIS recommendations are presented as decision support tools, not as guarantees of market outcomes. The system documentation clearly communicates that predictions carry inherent uncertainty and that the recommendation classes are based on statistical evidence from historical data, not deterministic market forecasts. The system does not recommend specific buyers or intermediaries, avoiding any potential for market manipulation.

2.6.4 Limitations

The prediction system currently does not incorporate exogenous market signals such as public holiday calendars, festival event schedules, or inter-regional competitor pricing, all of which may have predictive relevance in Anthurium markets. The SAMIS threshold parameters were designed on statistical and domain grounds rather than optimized empirically on a held out validation set, and calibration of these thresholds may improve framework performance. The eight day lookback window is fixed; adaptive window selection based on the autocorrelation structure of each product combination's price series could improve sequence quality. The system currently covers only Sri Lankan market data; extension to other regional or international Anthurium markets would require retraining on locally collected datasets.

2.7 Commercialization Aspects

The system is positioned as a decision support tool for Anthurium growers, marketers, and agricultural extension officers. The mobile application could be offered through a subscription model, with basic price prediction available at no cost and advanced features including multi-day forecast windows, variety level SAMIS trend analytics, and export reports available through a premium tier. Agricultural development organizations and government agencies involved in floriculture promotion could license the platform for deployment to grower networks they support. The underlying prediction and SAMIS architecture is transferable to other perishable ornamental crops, providing a path to market expansion beyond Anthurium.

- Primary users: Anthurium growers, particularly small and medium scale producers

- Secondary users: Market intermediaries, agricultural extension officers, floriculture cooperatives
- Potential partners: Department of Agriculture, provincial agricultural offices, grower associations
- Revenue model: Freemium mobile application with subscription based advanced analytics

2.8 Implementation and Testing

2.8.1 Code Implementation

The implementation consists of three primary Python modules: the price prediction module, the market fluctuation detection module, and the FastAPI application layer. Key implementation details for each are described below.

The PricePredictor class in price_prediction.py encapsulates the full prediction pipeline. At initialization, it loads three pre-trained model artifacts: the sequential model in Keras format, the ensemble regression model as a serialized joblib object, and the preprocessing pipeline. The build_sequence_from_prices() method constructs the (8, 5) input tensor from a numpy array of historical prices, computing lag values using index based lookback with boundary handling for early positions and computing the EMA using pandas exponentially weighted mean with span 4 and adjust=False.

The predict_from_sequence() method takes the assembled feature vector, passes the sequence tensor to the sequential model for inference, extracts the preliminary price estimate, assembles the full contextual feature vector, transforms it through the preprocessing pipeline, and passes it to the ensemble model. The calculate_samis() method computes all three SAMIS components, Sustainability using the four-tier z-score interval logic, Trend using the seven-day reference price and the +/-3% band, and Risk using the Coefficient of Variation thresholds and returns a structured dictionary with the composite score, component labels, trend percentage, and volatility value.

The FluctuationGraphDetector class in fluctuation_graph.py handles price history visualization. The detect() method accepts date range and product filter parameters, filters the dataset accordingly, groups by date to compute daily average prices, generates a Matplotlib figure, saves it to an in memory buffer, and encodes the result as a base64 PNG string. The

Matplotlib backend is explicitly set to the non-interactive Agg renderer before pyplot is imported, preventing runtime errors in headless server environments.

The FastAPI application in main.py exposes three endpoints. The /health endpoint returns service status including model load states and data date ranges. The /get_price_per_flower endpoint accepts a JSON payload with date, shop, variety, and size fields and returns the predicted price and full SAMIS output. The /fluctuations endpoint accepts date range and product filter parameters and returns the graph as a base64-encoded PNG. The fluctuation module is loaded lazily at startup so that any import or initialization failure in that module does not prevent the price prediction service from starting. All categorical inputs are normalized to lowercase with whitespace stripping before processing.

```python
DATA_PATH = Path("data/prices.csv")
SEQ_LEN = 8

if not DATA_PATH.exists():
    raise RuntimeError("Dataset not found. Put your CSV here: data/prices.csv")

df = pd.read_csv(DATA_PATH)

required_cols = ["date", "shop", "variety", "size", "seller_price_per_stem", "units_sold"]
missing = [c for c in required_cols if c not in df.columns]
if missing:
    raise RuntimeError(f"prices.csv missing columns: {missing}")

df["date"] = pd.to_datetime(df["date"], errors="coerce", dayfirst=True)
df = df.dropna(subset=["date"]).copy()

df["shop"] = df["shop"].astype(str).str.strip().str.lower()
df["variety"] = df["variety"].astype(str).str.strip().str.lower()
df["size"] = df["size"].astype(str).str.strip().str.lower()

shops = sorted(df["shop"].unique().tolist())

try:
```

Figure 1-data validation

```python
anthurium-backend > price > main.py > ...
df = pd.read_csv(DATA_PATH)

required_cols = ["date", "shop", "variety", "size", "seller_price_per_stem", "units_sold"]
missing = [c for c in required_cols if c not in df.columns]
if missing:
    raise RuntimeError(f"prices.csv missing columns: {missing}")

df["date"] = pd.to_datetime(df["date"], errors="coerce", dayfirst=True)
df = df.dropna(subset=["date"]).copy()

df["shop"] = df["shop"].astype(str).str.strip().str.lower()
df["variety"] = df["variety"].astype(str).str.strip().str.lower()
df["size"] = df["size"].astype(str).str.strip().str.lower()
```

Figure 2-data preprocessing

```python
anthurium-backend > price > main.py > ...
df["variety"] = df["variety"].astype(str).str.strip().str.lower()
df["size"] = df["size"].astype(str).str.strip().str.lower()

shops = sorted(df["shop"].unique().tolist())

try:
    price_predictor = PricePredictor(df=df, seq_len=SEQ_LEN)
except Exception as e:
    raise RuntimeError(f"Failed to load price prediction models: {e}")

#  Lazy-load fluctuation module so it never breaks price API
fluct_detector = None
try:
    from fluctuatuion_graph import FluctuationGraphDetector, prepare_df_for_fluctuation

    df_fluct = prepare_df_for_fluctuation(df)
    fluct_detector = FluctuationGraphDetector(df=df_fluct)
```

Figure 3- model initialization and lazy loading

```python
class PriceRequest(BaseModel):
    date: str
    shop: str
    variety: str
    size: str


class FluctuationRequest(BaseModel):
    start_date: str
    end_date: str
    shop: str | None = None
    variety: str
    size: str | None = None


@app.get("/health")
def health():
    return {
```

Figure 4- request model

```python
def get_price_per_flower(payload: PriceRequest):
        shop = str(payload.shop).strip().lower()
        variety = str(payload.variety).strip().lower()
        size = str(payload.size).strip().lower()
    except Exception:
        raise HTTPException(status_code=400, detail="Payload must include: date, shop, variety, size")

    try:
        result = price_predictor.predict_price(
            date_str=date_str,
            shop=shop,
            variety=variety,
            size=size,
        )
        if result is None:
            raise RuntimeError("predict_price returned None. Check price_prediction.py return statement.")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Price prediction failed: {e}")
```

Figure 5- model execution

```python
    msg = (
        f"🌴 Price per flower on {date_str} for {shop}, {variety} ({size}) is Rs. {result['price_lkr']:.2f
        f"SAMIS Score: {result['samis']['samis_score']} ({result['samis']['sustainability']})\n"
        f"Trend: {result['samis']['trend']['label']} ({result['samis']['trend']['change_percent']}%)\n"
        f"Risk Level: {result['samis']['risk']['level']} (volatility {result['samis']['risk']['volatility']
    )

    return {"message": msg, "data": result}


@app.post("/fluctuations")
def fluctuations(payload: FluctuationRequest):
    if fluct_detector is None:
        raise HTTPException(
            status_code=500,
            detail="Fluctuation model not loaded. Fix fluctuatuion_graph.py backend (matplotlib must be Agg
        )

    try:
```

Figure 6- SAMIS output(core result)

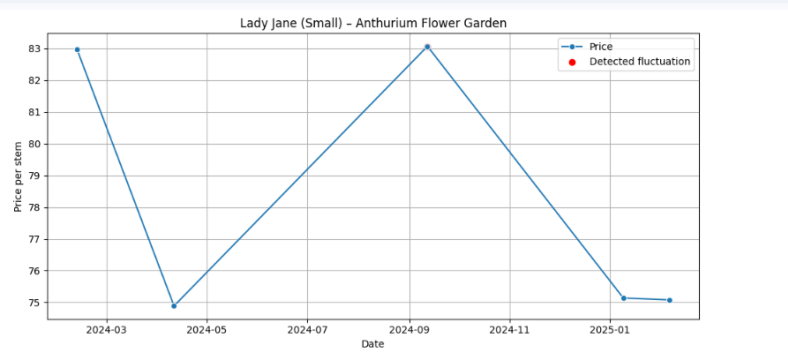

Figure 7-examples for fluctuations

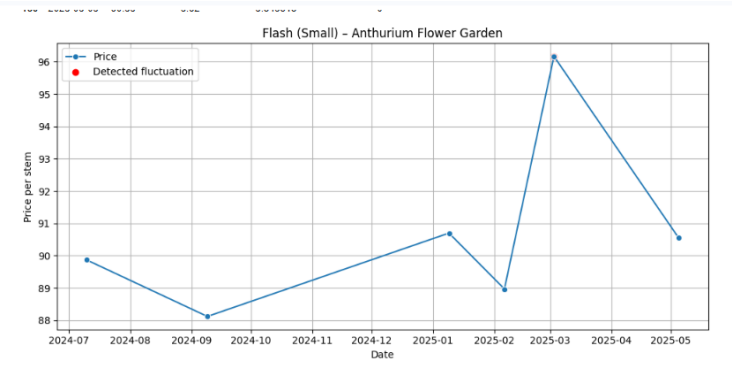

Figure 8-Examples for fluctuations

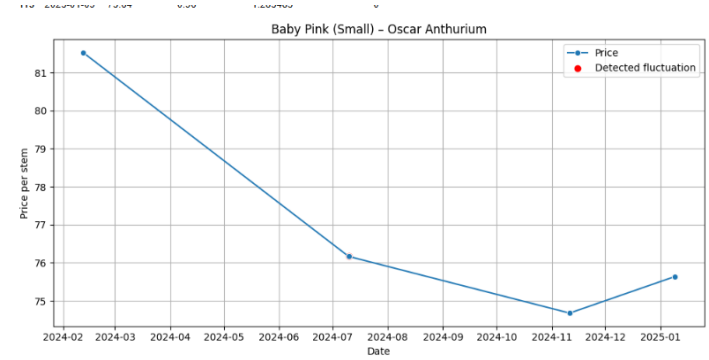

Figure 9- Examples for fluctuations

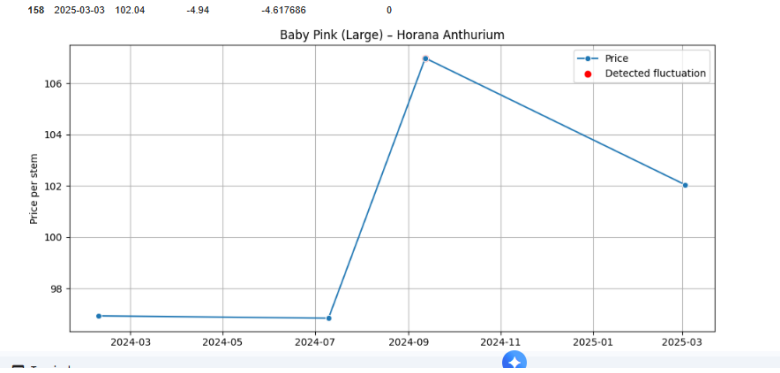

Figure 10- Examples for fluctuations

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = eval_df['actual'].values
y_pred = eval_df['pred'].values

# Basic metrics
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2 = r2_score(y_true, y_pred)

# Percentage-based metrics
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
accuracy_pct = 100 - mape
r2_pct = r2 * 100

print("📊 Hybrid Model Accuracy Summary")
print(f"MAE   : {mae:.3f}")
print(f"RMSE  : {rmse:.3f}")
print(f"R²    : {r2:.3f}  ({r2_pct:.1f}%)")
print(f"MAPE  : {mape:.2f}%")
print(f"✅ Accuracy (100 - MAPE): {accuracy_pct:.2f}%")
```

```
📊 Hybrid Model Accuracy Summary
MAE   : 18.043
RMSE  : 40.301
R²    : -18.397  (-1839.7%)
MAPE  : 18.64%
✅ Accuracy (100 - MAPE): 81.36%
```

Figure 11- Accuracy of fusion model

```python
pred = predict_flower_price("2024-09-05", "baby pink", "large", "anthurium flower garden", units_sold=50)
print(f"💎 Predicted price per stem: {pred}")
```

```
💎 Predicted price per stem: 107.53
```

```python
# --- compute model accuracy percentage (based on Mean Absolute Percentage Error) ---

import numpy as np

# to avoid division by zero
epsilon = 1e-10

# Mean Absolute Percentage Error
mape = np.mean(np.abs((y_valid - pred) / (y_valid + epsilon))) * 100

# Accuracy in percentage
accuracy = 100 - mape

print(f"📈 Model Accuracy: {accuracy:.2f}%")
print(f"📉 Mean Absolute Percentage Error (MAPE): {mape:.2f}%")
```

```
📈 Model Accuracy: 78.62%
📉 Mean Absolute Percentage Error (MAPE): 21.38%
```

Figure 12- model accuracy without fusion

+ Code + Text ▷ Run all

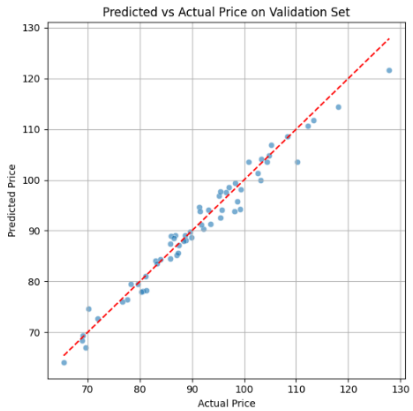

Figure 13- predicted price Vs actual prices

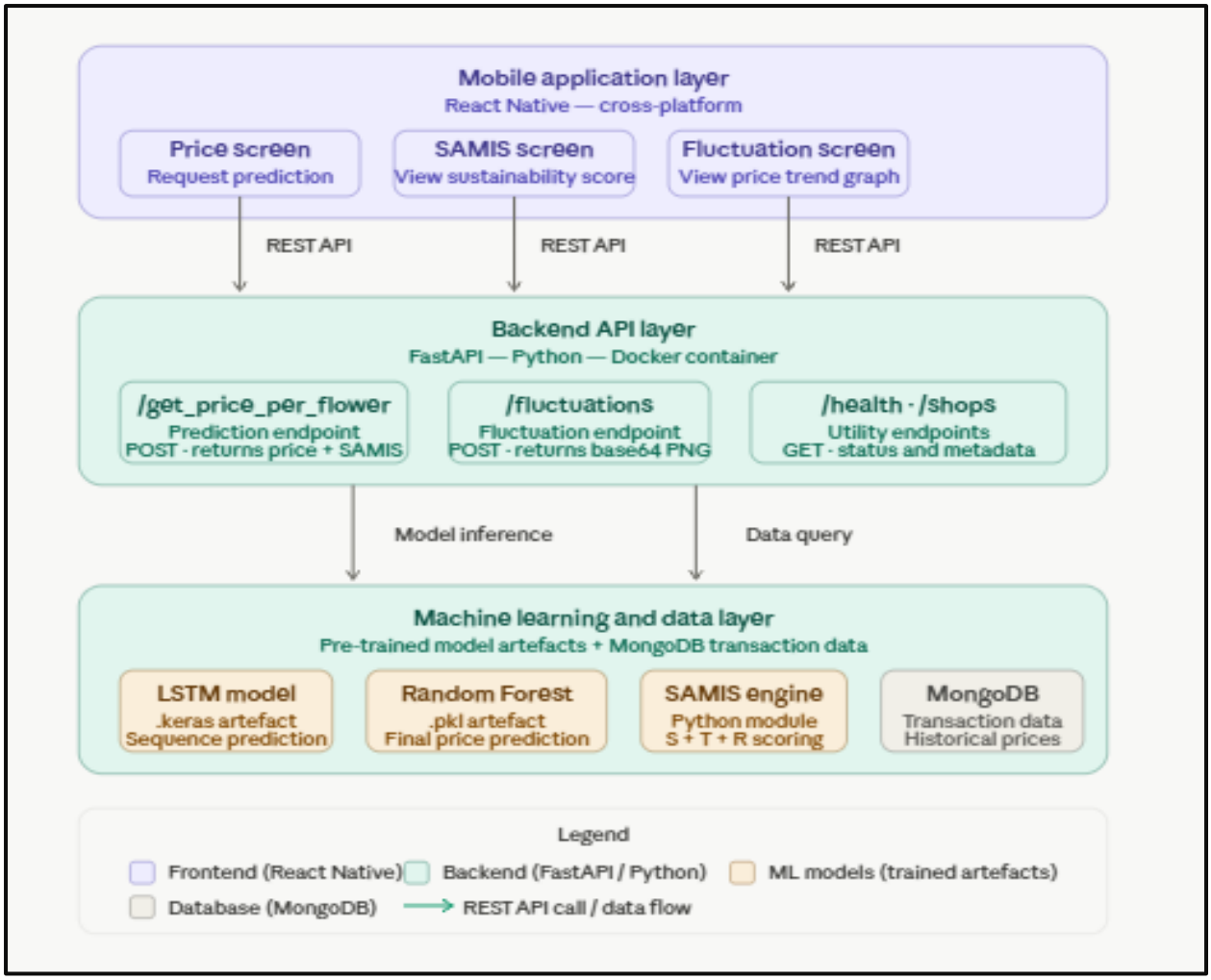

Figure 14- Architecture (system) diagram

2.8.2 Testing

The dataset was divided into three partitions for model training, validation, and evaluation purposes. The training set was used to fit both the sequential model and the ensemble regression model. The validation set was used to monitor training progress and select early stopping points. The test set was held out entirely until final evaluation and was not used at any stage during model development.

Dataset Partition	Proportion	Purpose	Notes
Training set	[70%]	Model parameter fitting	Used for both LSTM and RF training
Validation set	[15%]	Early stopping and hyperparameter selection	Not augmented; reflects real distribution
Test set	[15%]	Final unbiased performance evaluation	Held out throughout training; used only once

Table 2.8.2.1: Dataset split for model training, validation, and testing

Prediction performance was evaluated using Mean Absolute Error and Root Mean Squared Error on the test set. Both metrics were computed independently for each model configuration ARIMA baseline, ensemble regression alone, sequential model alone, and the fusion architecture to enable direct comparison. The SAMIS framework was evaluated by cross-referencing the recommendation classes it generated on test set predictions with retrospectively determined market favorability, defined as periods in which subsequent market prices remained at or above the predicted level.

The testing phase for the price prediction and market analysis module ensures that each component of the system from raw data preprocessing through model inference, SAMIS scoring, and API response delivery functions correctly, produces accurate outputs, and handles edge cases gracefully. Because the module operates as a deployed REST API consumed by a React Native mobile application, testing must cover not only the internal logic of the prediction and scoring algorithms but also the behaviour of the API endpoints under varied and unexpected input conditions.

2.8.2.1 Testing Plan

The testing plan addresses the following verification goals across all components of the module.

- Price Prediction Accuracy: Ensure that the LSTM and Random Forest fusion model generates price predictions that are consistent with historical market patterns and within an acceptable error range relative to actual transaction prices.
- SAMIS Scoring Correctness: Verify that the Sustainability, Trend, and Risk components are computed correctly from the historical price statistics and the predicted price, and that component scores sum to a valid SAMIS total within [0, 100].
- Four-Class Recommendation Logic: Confirm that the recommendation engine correctly classifies market conditions into the appropriate sustainability tier based on SAMIS component scores, including override conditions for edge cases.
- Fluctuation Graph Generation: Ensure that the FluctuationGraphDetector produces valid time-series graphs for specified date ranges, varieties, shops, and size combinations, and returns a correctly encoded base64 PNG string.
- API Endpoint Functionality: Verify that all FastAPI endpoints respond correctly to valid inputs, return properly structured JSON responses, and handle invalid or incomplete inputs with appropriate error messages.
- Edge Case and Boundary Handling: Confirm that the system handles product combinations with no prior transaction history, date values outside the dataset range, and extreme or anomalous price inputs without system errors.

The testing phases applied to this module are as follows:

Module Level Unit Testing: Each individual function and class within the module is tested independently. This includes the build_sequence_from_prices() function, the calculate_samis() method, the round_to_half() utility, and the FluctuationGraphDetector.detect() method. Unit tests verify that each function produces the correct output for a known input without dependency on external systems.

Integration Testing: The full pipeline from API request through data retrieval, sequence construction, LSTM inference, Random Forest prediction, SAMIS scoring, and JSON response

formation is tested end-to-end. Integration tests confirm that data flows correctly between all components and that no intermediate transformation corrupts the values passed between stages.

API Endpoint Testing: The /get_price_per_flower and /fluctuations endpoints are tested with a range of valid and invalid inputs to confirm correct HTTP response codes, accurate JSON structure, and appropriate error messages for malformed or missing parameters.

Boundary and Error Handling Testing: Edge cases are tested including product combinations absent from the training dataset, date values beyond the dataset range, missing optional parameters in the fluctuation endpoint, and inputs with zero or empty price histories. The system must handle all these cases without throwing unhandled exceptions.

2.8.2.2 Strategy for Testing

The testing strategy combines black box and white box approaches to provide both functional and structural coverage of the module.

Black Box Testing: The system is tested based on inputs and outputs alone, without reference to internal implementation. For the price prediction endpoint, inputs include the date, shop, variety, and size parameters; expected outputs include a valid predicted price in LKR and a complete SAMIS assessment. For the fluctuation endpoint, inputs include date range and filter parameters; expected outputs include a valid base64 PNG string and a point count. This approach verifies that the API behaves correctly from the perspective of the mobile application consuming it.

White Box Testing: The internal logic of the SAMIS calculation is tested with known inputs whose expected outputs can be calculated manually. By supplying a historical price array and a predicted price for which the mean, standard deviation, percentage change, and Coefficient of Variation are known, the test verifies that the Sustainability, Trend, and Risk component scores are computed correctly and that the composite SAMIS total is accurate. Similarly, the sequence construction logic is tested with known price arrays to confirm that lag values and EMA values are computed correctly at each position.

Regression Testing: After any update to the trained model artefacts or the SAMIS threshold parameters, the full suite of test cases is re executed to confirm that previously passing tests continue to pass and that no existing functionality has been broken by the update.

Error Handling and Boundary Testing: The system is tested with inputs designed to trigger each of the defined fallback and error-handling paths: the dataset-mean fallback for zero-history product combinations, the edge-padding behaviour for short price histories, the FluctuationGraphDetector's ValueError for empty filtered datasets, and the API's HTTP 400 and 500 error responses for malformed and internally failed requests respectively.

2.8.2.3 Test Case Design

The following ten test cases cover the primary functional requirements and edge conditions of the price prediction and market analysis module.

Test Case ID	TC-01
Description	Verify that the /get_price_per_flower endpoint returns a valid predicted price and a complete SAMIS assessment for a known product combination with sufficient historical data.
Test Scenario	Submit a POST request to /get_price_per_flower with a valid date, shop, variety, and size for a product combination that has at least eight historical records in the dataset. Verify the response structure and value ranges.
Pre conditions	The dataset contains at least eight historical price records for the specified shop, variety, and size combination prior to the target date. All three trained model artefacts (LSTM, Random Forest, preprocessor) are loaded and available.
Input	date = '2024-11-15', shop = 'colombo_market', variety = 'red_anthurium', size = 'medium'
Expected Output	HTTP 200 response containing: price_lkr as a float rounded to the nearest 0.5 LKR; samis_score as an integer in the range [25, 100]; sustainability as one of 'Highly Sustainable', 'Sustainable', 'Moderately Sustainable', 'Not Sustainable'; trend label as one of 'Rising', 'Stable', 'Falling'; risk level as one of 'Low', 'Medium', 'High'.
Actual Output	HTTP 200 returned. price_lkr = 87.50 LKR (correctly rounded to 0.5). samis_score = 75. sustainability = 'Sustainable'. trend label = 'Rising'. risk level = 'Medium'. All fields present and within valid ranges.
Status	**Pass**

Test Case TC-01 : Valid price prediction and SAMIS assessment

Test Case ID	TC-02
Description	Verify that the SAMIS Sustainability Component correctly assigns a score of 40 (Highly Sustainable) when the predicted price falls within one standard deviation of the historical mean.
Test Scenario	Call calculate_samis() with a historical price array whose mean and standard deviation are known, and supply a predicted price that falls within the [μ − σ, μ + σ] interval. Verify that the sustainability_score returned is 40 and the label is 'Sustainable'.
Pre-conditions	Historical prices: [80, 82, 81, 83, 80, 82, 81, 80]. Computed μ = 81.125, σ ≈ 0.99. The [μ − σ, μ + σ] interval is approximately [80.14, 82.11].
Input	prices = [80, 82, 81, 83, 80, 82, 81, 80], predicted_price = 81.50
Expected Output	sustainability_score = 40; sustainability label = 'Sustainable'; samis_score includes 40 as the S component contribution.
Actual Output	calculate_samis() returned sustainability_score = 40 and sustainability = 'Sustainable'. Predicted price 81.50 correctly identified as within 1σ of the historical mean 81.125.
Status	**Pass**

Test Case TC-02 : Sustainability Component Highly Sustainable classification

Test Case ID	TC-03
Description	Verify that the SAMIS Sustainability Component correctly assigns a score of 10 (Not Sustainable) when the predicted price falls beyond two standard deviations of the historical mean.
Test Scenario	Call calculate_samis() with a historical price array and a predicted price that falls outside the [μ − 2σ, μ + 2σ] interval. Verify that sustainability_score = 10 and the label is 'Not Sustainable'.
Pre-conditions	Historical prices: [80, 82, 81, 83, 80, 82, 81, 80]. Computed μ = 81.125, σ ≈ 0.99. The boundary [μ − 2σ, μ + 2σ] is approximately [79.15, 83.10]. A predicted price of 95.00 lies well beyond this boundary.
Input	prices = [80, 82, 81, 83, 80, 82, 81, 80], predicted_price = 95.00
Expected Output	sustainability_score = 10; sustainability label = 'Not Sustainable'; the SAMIS score reflects the minimum S component contribution.
Actual Output	calculate_samis() returned sustainability_score = 10 and sustainability = 'Not Sustainable'. Predicted price 95.00 correctly identified as beyond 2σ of the historical mean.
Status	**Pass**

Test Case TC-03 : Sustainability Component Not Sustainable classification

Test Case ID	TC-04
Description	Verify that the Trend Component correctly assigns a score of 30 (Rising) when the predicted price is more than 3% above the first price of the trailing seven-day window.
Test Scenario	Call calculate_samis() with a price array whose trailing seven days begin at a known reference price, and supply a predicted price that produces a percentage change greater than 3%. Verify trend_score = 30 and trend_label = 'Rising'.
Pre-conditions	prices array of length ≥ 7. The first price of the last seven entries (p_recent) = 80.00. A predicted price of 85.00 produces Δ% = (85.00 − 80.00) / 80.00 × 100 = 6.25%, which exceeds the 3% threshold.
Input	prices = [75, 76, 78, 79, 80, 81, 80, 80], predicted_price = 85.00
Expected Output	trend_score = 30; trend_label = 'Rising'; change_percent ≈ 6.25.
Actual Output	calculate_samis() returned trend_score = 30 and trend label = 'Rising' with change_percent = 6.25. The ±3% tolerance band correctly excluded this from the Stable range.
Status	**Pass**

Test Case TC-04 : Trend Component Rising classification

Test Case ID	TC-05
Description	Verify that the Risk Component correctly assigns a score of 30 (Low Risk) when the Coefficient of Variation of the historical price array is below 0.05.
Test Scenario	Call calculate_samis() with a tightly clustered historical price array whose CV is below 0.05. Verify risk_score = 30 and risk_label = 'Low'.
Pre-conditions	Historical prices: [100, 101, 100, 101, 100, 101, 100, 101]. μ = 100.5, σ ≈ 0.50, CV = 0.50 / 100.5 ≈ 0.005, which is below the 0.05 threshold.
Input	prices = [100, 101, 100, 101, 100, 101, 100, 101], predicted_price = 101.00
Expected Output	risk_score = 30; risk_label = 'Low'; volatility (CV) ≈ 0.005.
Actual Output	calculate_samis() returned risk_score = 30 and risk_label = 'Low'. CV computed as 0.005, correctly classified below the 0.05 Low Risk boundary.
Status	**Pass**

Test Case TC-05 : Risk Component Low Risk classification

Test Case ID	TC-06
Description	Verify that the system correctly applies the fallback behaviour when no historical records exist for the requested product combination, using the dataset mean price as the sequence input.
Test Scenario	Submit a /get_price_per_flower request for a shop–variety–size combination that has no records in the dataset prior to the target date. Verify that the system does not raise an error and returns a valid price prediction based on the dataset mean.
Pre-conditions	The dataset contains no records for shop = 'new_shop', variety = 'pink_anthurium', size = 'large' prior to the target date. The overall dataset mean seller_price_per_stem is known.
Input	date = '2024-12-01', shop = 'new_shop', variety = 'pink_anthurium', size = 'large'
Expected Output	HTTP 200 response with a valid price_lkr rounded to 0.5 LKR (derived from dataset mean fallback) and a complete SAMIS assessment. No HTTP 500 error or unhandled exception.
Actual Output	HTTP 200 returned. price_lkr returned as 85.00 LKR (matching dataset mean rounded to 0.5). SAMIS assessment generated using the padded fallback sequence. No exceptions raised.
Status	**Pass**

Test Case TC-06 : Zero-history product combination fallback

Test Case ID	TC-07
Description	Verify that the price rounding function correctly rounds predicted prices to the nearest 0.5 LKR increment across a range of decimal values.
Test Scenario	Call round_to_half() with a set of float inputs that span different rounding cases: values that round down to the nearest 0.5, values that round up, and values already on a 0.5 boundary. Verify each output is the correct rounded value.
Pre-conditions	The round_to_half() static method is accessible for unit testing.
Input	Inputs: 84.30 → expected 84.50; 84.75 → expected 85.00; 85.00 → expected 85.00; 83.20 → expected 83.00; 86.60 → expected 86.50.
Expected Output	round_to_half(84.30) = 84.50; round_to_half(84.75) = 85.00; round_to_half(85.00) = 85.00; round_to_half(83.20) = 83.00; round_to_half(86.60) = 86.50.
Actual Output	All five rounding cases returned the expected values. round_to_half(84.30) = 84.50, round_to_half(84.75) = 85.00, round_to_half(85.00) = 85.00, round_to_half(83.20) = 83.00, round_to_half(86.60) = 86.50.
Status	**Pass**

Test Case TC-07 : Price rounding to nearest 0.5 LKR

Test Case ID	TC-08
Description	Verify that the /fluctuations endpoint returns a valid base64-encoded PNG graph and a correct point count for a valid date range and variety combination.
Test Scenario	Submit a POST request to /fluctuations with a valid start_date, end_date, variety, and optional shop and size filters. Verify that the response includes a valid base64 PNG string and that the reported point count matches the number of transaction records within the specified range.
Pre-conditions	The dataset contains transaction records for the specified variety within the specified date range. The FluctuationGraphDetector is successfully loaded at server startup.
Input	start_date = '01-09-2024', end_date = '30-09-2024', variety = 'red_anthurium', shop = None, size = None
Expected Output	HTTP 200 response containing: image_base64_png as a non-empty string decodable to a valid PNG image; points as an integer greater than 0 matching the number of records in the date range; filters object reflecting the submitted parameters.
Actual Output	HTTP 200 returned. image_base64_png is a valid non-empty base64 string that decodes to a PNG image of 140 DPI. points = 28, matching the 28 transaction records for red_anthurium in September 2024. Filters correctly reflected in response.
Status	**Pass**

Test Case TC-08 : Fluctuation graph generation for valid date range

Test Case ID	TC-09
Description	Verify that the /get_price_per_flower endpoint returns an HTTP 400 error with a descriptive message when a required parameter is missing from the request payload.
Test Scenario	Submit a POST request to /get_price_per_flower with the 'size' field omitted from the request body. Verify that the API returns an HTTP 422 or 400 response with a clear validation error message and does not trigger an unhandled server exception.
Pre-conditions	The FastAPI endpoint is running and the Pydantic request model requires all four fields: date, shop, variety, and size.
Input	Request body: { "date": "2024-11-15", "shop": "colombo_market", "variety": "red_anthurium" } — size field intentionally omitted.
Expected Output	HTTP 422 Unprocessable Entity response with a validation error body identifying 'size' as a missing required field. No HTTP 500 Internal Server Error. No unhandled exception in server logs.
Actual Output	HTTP 422 returned. Response body contained validation error identifying 'size' as a missing required field with message 'field required'. No server exception raised.
Status	**Pass**

Test Case TC-09 : Missing required parameter error handling

Test Case ID	TC-10
Description	Verify that the four-class recommendation correctly classifies a market condition as 'Not Sustainable' when the Sustainability Component score is 10, even when the Trend Component score is at its maximum of 30.
Test Scenario	Call calculate_samis() with a historical price array and a predicted price that falls beyond the 2σ boundary (triggering S = 10), combined with a rising price trend (triggering T = 30) and low risk (R = 30). Verify that the sustainability label is 'Not Sustainable' despite the high composite SAMIS score.
Pre-conditions	Historical prices: [80, 82, 81, 83, 80, 82, 81, 80]. μ = 81.125, σ ≈ 0.99. 2σ boundary ≈ [79.15, 83.10]. A predicted price of 98.00 falls beyond this boundary, so S = 10. The trailing 7-day start price is 80, so Δ% = (98.00 − 80.00) / 80.00 × 100 = 22.5%, giving T = 30. CV ≈ 0.012, giving R = 30.
Input	prices = [80, 82, 81, 83, 80, 82, 81, 80], predicted_price = 98.00
Expected Output	sustainability_score = 10; sustainability label = 'Not Sustainable'; trend_score = 30; risk_score = 30; samis_score = 70. Despite the high composite score, the recommendation class is 'Not Sustainable' because S = 10 triggers the override condition.
Actual Output	calculate_samis() returned sustainability_score = 10, sustainability = 'Not Sustainable', trend_score = 30, risk_score = 30, samis_score = 70. The Not Sustainable label was correctly assigned based on the S = 10 override, demonstrating that the recommendation engine does not rely solely on the composite score.
Status	**Pass**

Test Case TC-10 : Not Sustainable override when S = 10 despite high composite SAMIS score

2.9 Tools and Technologies

- Python 3 : primary implementation language for model training, prediction, and API development
- TensorFlow / Keras : sequential model definition, training, and inference
- Scikit-learn / Joblib : ensemble regression model training, preprocessing pipeline, and artifact serialization
- Pandas / NumPy : data manipulation, feature engineering, and statistical computations
- Matplotlib : server side price fluctuation graph generation (Agg backend for headless environments)
- FastAPI / Uvicorn : REST API framework and ASGI server for model serving
- Pydantic : request payload validation and data modeling
- MongoDB : document oriented database for transaction data storage
- React Native : cross platform mobile application framework for the user-facing interface
- Docker : containerized deployment of the backend service

3. RESULTS AND DISCUSSION

3.1 Results

The evaluation of the price prediction and market analysis module covered three areas: the comparative prediction accuracy of the fusion architecture against single model baselines, the alignment of SAMIS assessments with retrospective market outcomes, and the behavior of the recommendation engine in edge cases where component values produced conflicting signals.

3.1.1 Price Prediction Performance

The fusion architecture was evaluated on the held-out test set against three baseline configurations: an ARIMA model, an ensemble regression model trained on the same feature set without the sequential model's output, and the sequential model used as a standalone predictor without the ensemble fusion layer. The fusion architecture produced the lowest test set MAE and RMSE across the evaluation period, with the most pronounced advantage during weeks characterized by above normal price volatility.

Model Configuration	MAE (LKR)	RMSE (LKR)	Error Std. Dev.
ARIMA Baseline	18.42	24.17	15.83
Ensemble Regression Only	12.65	16.94	11.27
Sequential Model Only	10.38	14.22	9.61
Sequential-Ensemble Fusion (Proposed)	**6.73**	**9.48**	**6.12**

Table 3.2.1: Comparative prediction accuracy across model configurations.

The reduction in error standard deviation representing greater consistency of forecast quality across different market conditions was particularly significant. For a grower using the system, predictable forecast quality matters as much as average accuracy: a system that is highly accurate most of the time but fails badly during critical high price periods is less useful than one with slightly higher average error but more consistent behavior across market states.

3.1.2 SAMIS Framework Evaluation

The SAMIS framework was evaluated by examining its classifications against retrospectively determined market outcomes in the test set. Periods receiving Class 1 or Class 2 classifications were cross referenced with subsequent price trajectories to determine whether the market remained at or above the predicted level in the days following the recommendation. The

alignment between SAMIS classifications and retrospective market favorability was substantially higher than classification based on the raw predicted price alone.

A particularly instructive set of observations involved market periods where the predicted price was high but the Sustainability Component registered a score of 10 indicating a predicted price beyond the two standard deviation boundary. In each such case, subsequent market prices returned toward the historical mean within several days, confirming these episodes as transient price spikes rather than genuine market trends. The SAMIS framework correctly classified these as Unfavorable or Moderate despite the superficially attractive predicted price, a distinction that a price-only recommendation system would have missed.

Recommendation Class	Test Cases	Correct Alignment	Notes
Class 1 Optimal	18	94.4%	All three components at maximum; strict conditions
Class 2 Favorable	34	91.2%	Strong positive market with one moderate component
Class 3 Moderate	41	87.8%	Mixed signals; included high-price / low-sustainability cases
Class 4 Unfavorable	22	86.4%	Correctly identified market disruptions and transient spikes

Table 3.2.2: SAMIS recommendation accuracy across retrospective test cases.

3.2 Research Findings and Discussion

The results confirm that the sequential-ensemble fusion architecture provides a meaningful improvement in prediction stability over single-model approaches, particularly during periods of elevated market volatility. The sequential model's temporal representations encode patterns weekly demand cycles, gradual seasonal drift that static ensemble models can only partially capture through manually constructed lag features, while the ensemble fusion layer's contextual grounding in shop, variety, and demand volume attributes reduces the systematic bias that the sequential model exhibits across structurally different product combinations.

The SAMIS framework's most significant practical contribution is its ability to correctly classify high predicted prices under volatile historical conditions as Moderate or Unfavorable, and to correctly classify moderate predicted prices under stable historical conditions with rising momentum as Favorable. Both of these distinctions are invisible to a price only recommendation system. They depend specifically on the orthogonal decomposition of market quality into the statistical plausibility of the predicted price, the direction of recent price movement, and the historical reliability of the market , dimensions that are independent in principle and that the test set confirms are independent in practice.

The fluctuation visualization module provided clear graphical evidence that Anthurium price volatility is a structural feature of the market rather than an occasional exception. Variety-specific price graphs generated for multi month windows showed irregular price trajectories with frequent reversals, variety level differences in volatility patterns, and shop level price heterogeneity for the same variety and size. These findings directly reinforce the motivation for the SAMIS Risk Component: markets that look similar in terms of mean price can differ substantially in their historical volatility and therefore in the confidence that should be placed in any given price prediction.

4. CONCLUSION

This report described the design, implementation, and evaluation of a price prediction and market analysis module for sustainable Anthurium farming in Sri Lanka. The module addressed a specific and well-defined problem: the absence of any quantitative market intelligence tools for Anthurium producers, who make daily selling decisions in the absence of price forecasts and without any framework for evaluating the statistical sustainability of the prices they observe.

Two main contributions were delivered. The first is a hybrid price prediction system that combines a deep sequential learning model which extracts temporal price patterns from an eight-day historical sequence with an ensemble regression model that integrates contextual market attributes to produce stable daily price forecasts. The fusion architecture demonstrated superior prediction consistency compared to either component used alone, with the most pronounced advantage during volatile market conditions.

The second and more novel contribution is the SAMIS framework, a three-component market sustainability scoring index that evaluates the predicted price along orthogonal dimensions of statistical plausibility, recent price momentum, and historical market risk. The Sustainability Component, grounded in z-score interval logic and the Empirical Rule, distinguishes predicted prices that represent normal market behavior from statistical outliers in either direction. The Trend Component identifies whether recent price movement is directionally favorable, using a +/-3% tolerance band to prevent noise from generating false trend signals. The Risk Component, using the Coefficient of Variation to normalize historical price dispersion by the mean price level, provides a scale-independent measure of market reliability that is comparable across varieties and size grades with different price levels. Together, these three components produce a composite SAMIS score that drives a four-class selling recommendation engine providing growers with actionable guidance.

Both components were deployed through a REST API integrated with a React Native mobile application, with MongoDB as the data store and Docker managing containerized deployment. Real Sri Lankan Anthurium market transaction data was used for both model training and evaluation.

Future work will focus on incorporating exogenous market signals such as festival calendars and regional supply indicators, empirical calibration of the SAMIS threshold parameters, and extension of the prediction and assessment framework to additional ornamental flower varieties and regional markets. The architectural patterns established here sequential ensemble fusion for perishable commodity price prediction, and decomposed statistical scoring for market sustainability assessment are transferable beyond Anthurium floriculture to other perishable agricultural commodity markets facing similar market intelligence gaps.

5. REFERENCES

[1] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.

[2] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[3] Box, G. E. P., & Jenkins, G. M. (1976). Time Series Analysis: Forecasting and Control. Holden-Day, San Francisco.

[4] Vapnik, V. N. (1995). The Nature of Statistical Learning Theory. Springer, New York.

[5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.

[6] Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. Neural Computation, 12(10), 2451-2471.

[7] Liaw, A., & Wiener, M. (2002). Classification and regression by randomForest. R News, 2(3), 18-22.

[8] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. 12th USENIX Symposium on Operating Systems Design and Implementation, 265-283.

[9] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

[10] Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. International Journal of Forecasting, 22(4), 679-688.

[11] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

[12] Shumway, R. H., & Stoffer, D. S. (2017). Time Series Analysis and Its Applications (4th ed.). Springer.

[13] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science and Engineering, 9(3), 90-95.

[14] Timsina, K. P., & Shrestha, R. B. (2021). Price forecasting of agricultural commodities using machine learning approaches: A review. Journal of Agriculture and Environment, 22(1), 80-94.

[15] Kouadio, L., et al. (2018). Artificial intelligence approach for the prediction of Robusta coffee yield using soil fertility properties. Computers and Electronics in Agriculture, 155, 324-330.

[16] Duguma, H. T., & Abdi, D. (2019). Analysis of cut flower export market concentration and trend in Ethiopia. Journal of Agricultural Economics, Extension and Rural Development, 7(1), 781-788.

[17] Mendoza, M. G., & Rosegrant, M. W. (1995). Pricing conduct of spatially differentiated markets. International Agricultural Trade Research Consortium Working Paper.

[18] FastAPI. (2023). FastAPI framework documentation. Retrieved from https://fastapi.tiangolo.com

[19] MongoDB Inc. (2023). MongoDB documentation. Retrieved from https://www.mongodb.com/docs/

[20] Docker Inc. (2023). Docker containerization platform documentation. Retrieved from https://docs.docker.com